CS 5326: Advanced Generative AI and Agents

Programming Assignment 3:
Search over Reasoning and Step-Level Process Reward Models

Design and Lead: Muhammad Ahmad Sarfraz

Fall 2026

Abstract

While pre-training instills base language capabilities (PA1) and post-training aligns models with human preferences (PA2), modern frontier systems increasingly rely on **test-time compute** and **deliberate search over reasoning**. This assignment explores inference-time scaling for multi-step deductive problem solving without modifying model weights. Working with an asymmetric policy-verifier pair on Kaggle GPU T4x2, you will deploy a 1.5B generator (`Qwen/Qwen2.5-1.5B-Instruct` in float16) and a 7B verifier (`Qwen/Qwen2.5-7B-Instruct` in 4-bit NF4) to construct an end-to-end inference-time search engine over chain-of-thought trajectories. You will implement step-structured generation, greedy decoding, and self-consistency majority-voting baselines. You will then develop a **Process Reward Model (PRM)** stand-in using step-level binary verification, design aggregation strategies to score multi-step deduction paths, and implement **PRM-guided Best-of-N search**. Finally, you will conduct a rigorous empirical evaluation over a fixed 50-problem benchmark from GSM8K under both candidate-matched ($N = 5$) and compute-matched ($N_{\text{SC}} \approx 15$) budgets, analyzing where step verification succeeds, where automated judges hallucinate, and the computational efficiency of search versus sampling.

A Note from the Lead TA

The pragmatic use of Generative AI is intentionally allowed for this assignment, and we encourage using it as a modern productivity tool. But please do not just *"one-shot"* the problems for the sake of getting them done. You are among the most capable and competent minds in this country; do not let outsourcing your thinking undermine your own learning.
Inference-time search and step-level verifiers are at the absolute frontier of AI right now. Take the time to get your hands dirty: inspect the reasoning traces, see where the automated judges trip up, and understand why things break. Use AI to assist with debugging and syntax, but make sure the intuition, analysis, and engineering understanding remain genuinely yours.

Contents

1 Assignment Overview **3**
1.1 What You Will Implement . . . 3
1.2 What You Will Run and Evaluate . . . 3
1.3 How the Assignment is Graded . . . 4

2 Theoretical Background: Test-Time Compute & Search **5**
2.1 Inference-Time Scaling and the Test-Time Compute Frontier . . . 5
2.2 Outcome Reward Models (ORM) vs. Process Reward Models (PRM) . . . 5
2.3 Why Binary Step Verification? . . . 6
2.4 Path Score Aggregation Strategies . . . 6

3 Model Choice & Compute Environment **7**
3.1 The Local Track: Asymmetric Policy-Verifier Pair . . . 7
3.1.1 Technical Rationale . . . 7
3.2 Clone the Starter Repository . . . 7
3.3 Starter Repository Layout . . . 7
3.4 Kaggle Setup and Environment . . . 8

4 The Benchmark Dataset & Step-Structured Formatting **10**
4.1 The GSM8K 50-Problem Subset . . . 10
4.2 Step-Structured Prompt Schema . . . 10
4.3 Step and Answer Extraction Specifications . . . 10

5 Task Specifications & Problem Deliverables **11**
5.1 Problem 1: Step-Structured Generation & Baseline Decoders (20 Points) . . . 11
5.2 Problem 2: Process Reward Model (PRM) Verifier (25 Points) . . . 12
5.3 Problem 3: PRM-Guided Best-of-N Search & Budget Bookkeeping (20 Points) . . . 14
5.4 Problem 4: Evaluation, Comparative Analysis & Report (35 Points) . . . 15

6 Private Evaluation Interface: Submission Adapter Contract **17**
6.1 Running the Public Adapter Tests . . . 18

7 Technical Report Specifications **19**
7.1 Section 1: Benchmark Results and Scaling Curves (7 Points) . . . 19
7.2 Section 2: Path Score Aggregation Ablation (6 Points) . . . 19
7.3 Section 3: Failure Mode and Judge Error Analysis (7 Points) . . . 19
7.4 Section 4: Test-Time Compute Efficiency Analysis (3 Points) . . . 19
7.5 Section 5: Reproducibility and Limitations (2 Points) . . . 19

8 Grading Rubric Breakdown **20**

9 Submission Instructions **21**

10 Academic Integrity **21**

11 References **21**

1 Assignment Overview

1.1 What You Will Implement

1. **Step-Structured Reasoning Interface:** A zero-shot prompting schema enforcing discrete, numbered reasoning steps and a clean termination delimiter, accompanied by a resilient regular-expression answer parser.
2. **Baseline Search Strategies:**
   - *Greedy Baseline ($N = 1$):* Deterministic generation with `do_sample=False`.
   - *Self-Consistency Baseline ($N = 5$):* Stochastic temperature-sampled reasoning paths aggregated via outcome majority voting.
3. **Process Reward Model (PRM) Verifier:** An automated step-by-step judge (`Qwen2.5-7B-Instruct` in 4-bit NF4) evaluating individual step validity conditioned on the problem statement and prefix under local validity semantics.
4. **Path Aggregation Operators:** Mathematical aggregation functions (min-pooling, product of validity, and step error penalties) that reduce step-level verifier feedback into a scalar trajectory reward.
5. **PRM-Guided Best-of-N Search ($N = 5$):** A decoding-time search algorithm that ranks sampled reasoning paths using process rewards and breaks score ties deterministically via majority voting.
6. **Compute Bookkeeping, Scaling Curves, and Budget Matching:** Complete token- and call-level accounting that matches total test-time compute across search and pure sampling ($N_{\text{SC}} \approx 15$), along with a full self-consistency scaling curve over $N \in \{1, \ldots, 15\}$.

1.2 What You Will Run and Evaluate

1. Benchmark evaluation over 50 fixed math reasoning problems from the GSM8K test set (`data/gsm8k_test_50.jsonl`).
2. Controlled comparison across six programmatic configurations:
   - Baseline 1: Greedy Decoding ($N = 1$).
   - Baseline 2: Self-Consistency ($N = 5$, candidate-matched).
   - Baseline 3: Compute-Matched Self-Consistency ($N_{\text{SC}} \approx 15$).
   - Main Method 1: PRM-Guided Best-of-5 Search (min-pooling).
   - Main Method 2: PRM-Guided Best-of-5 Search (Product aggregation).
   - Main Method 3: PRM-Guided Best-of-5 Search (Step Penalty aggregation).
3. Self-consistency scaling curve over $N \in \{1, \ldots, 15\}$ serialized to `outputs/sc_scaling_curve.json`.
4. Systematic ablation of path aggregation strategies serialized to `outputs/aggregation_ablation.json`.
5. Failure-mode and error taxonomy analysis isolating judge false positives, judge false negatives, and shared generator errors.

Pedagogical Workload and Compute Feasibility

Unlike PA1 (which trained a Transformer from scratch) and PA2 (which fine-tuned models via DPO on dual GPUs), **PA3 requires no parameter updates, no backpropagation, and no multi-GPU memory synchronization**. Every operation is purely inference-time generation and scoring.
The entire local benchmark run takes **roughly 1 to 2 hours on Kaggle (T4x2), depending on batching** when using the dual-GPU T4x2 setup. The cognitive effort is centered on **algorithm design, verifier prompting, aggregation math, and empirical analysis**, not

pipeline plumbing.

1.3 How the Assignment is Graded

Grading is divided into an auto-graded programming contract, programmatic evaluation artifacts, and a rigorous technical report:

- **Auto-Graded Unit and Implementation Tests (65%):** Evaluated via private test harnesses through the documented `submission_adapter.py` contract. Your code must strictly conform to expected signatures and return types.
- **Evaluation Artifacts and Technical Report (35%):** Evaluated from your programmatic evaluation artifacts (10 pts for `outputs/results_summary.json`, `outputs/aggregation_ablation.json`, `outputs/sc_scaling_curve.json`, `outputs/per_problem.jsonl`, and `run_pipeline.ipynb` or `run_all.py`) and your written PDF report (25 pts for `REPORT.pdf`, maximum 6 pages). The report must document empirical findings, scaling curves, aggregation ablations, and a qualitative breakdown of judge behavior.

Library Boundary: What You Can and Cannot Use

Permitted: You may use standard PyTorch, Hugging Face `transformers`, `accelerate`, and `bitsandbytes` for model loading and local quantized inference. You may use standard data science utilities (`numpy`, `tqdm`, `re`, `json`).
Forbidden: You may **not** use third-party test-time search frameworks (e.g., LangChain, LlamaIndex, DSPy), external inference servers or engines (e.g., vLLM, TGI, SGLang), or prebuilt PRM libraries. All step prompts, answer parsers, verifier routines, aggregation formulas, majority-voting decoders, and search loops must be implemented in your own codebase from scratch using Hugging Face `.generate()`.

2 Theoretical Background: Test-Time Compute & Search

2.1 Inference-Time Scaling and the Test-Time Compute Frontier

Scaling pre-training compute (C_{train}) yields steady power-law improvements in language model capabilities (Kaplan et al., 2020; Hoffmann et al., 2022). However, once weights are frozen, the traditional paradigm generates completions via standard autoregressive greedy or nucleus decoding:

$$y_t \sim \pi_\theta(y_t \mid x, y_{<t}). \tag{1}$$

For complex, multi-step deductive problems (e.g., Olympiad mathematics, competitive coding, formal logic), standard left-to-right generation exhibits a compounding error probability: if step k introduces a subtle algebraic fallacy, every subsequent token $y_{>k}$ conditions on a flawed premise, doomed to arrive at an incorrect conclusion.

Recent research reveals that **allocating additional compute at inference time** (C_{test}) provides a complementary scaling axis (Brown et al., 2024; Snell et al., 2024). Rather than committing greedily to a single trajectory, an agent can explore multiple paths in the reasoning space, verify intermediate deductions, and search for the most logically coherent solution.

2.2 Outcome Reward Models (ORM) vs. Process Reward Models (PRM)

When searching through reasoning trajectories, how should we evaluate candidates? Consider a math problem where a shirt costs \$12 and a pair of pants costs 3 times the price of a shirt (\$36), and we want the total cost for 2 shirts and 1 pair of pants (correct total: \$60).

(a) Outcome Reward Model (ORM)	**(b) Process Reward Model (PRM)**	
Step 1: Pants cost $3 \times 12 = 36$.	Step 1: Pants cost $3 \times 12 = 36$.	$\longrightarrow \mathbf{r_1 = +1.0}$
$\downarrow$	$\downarrow$	
Step 2: 2 shirts cost $2 \times 12 = 28$. **(Flaw!)**	Step 2: 2 shirts cost $2 \times 12 = 28$.	$\longrightarrow \mathbf{r_2 = -1.0}$ **(Flaw!)**
$\downarrow$	$\downarrow$	
Step 3: Total cost is $28 + 36 = 60$. **(Flaw!)**	Step 3: Total cost is $28 + 36 = 60$.	$\longrightarrow \mathbf{r_3 = -1.0}$
$\downarrow$	$\downarrow$	
Answer: 60 $\longrightarrow \mathbf{R_{ORM} = +1.0}$ (False Positive)	Answer: 60	$\longrightarrow$ **Path Rejected**

Figure 1: Comparison of ORM vs. PRM on a compensating-error trajectory. The problem asks for the cost of 2 shirts (\$12 each) and 1 pair of pants (\$36). The flawed derivation makes two compensating arithmetic mistakes (Step 2 computes $2 \times 12 = 28$; Step 3 computes $28 + 36 = 60$), accidentally arriving at the correct final answer of 60. An ORM inspects only the final outcome and rewards the flawed path ($R_{\text{ORM}} = +1.0$). A PRM verifies each deduction locally, catching the error at Step 2 and rejecting the trajectory.

1. **Outcome Reward Models (ORM):** An ORM scores only the final generated output:
$$R_{\text{ORM}}(\tau) = f_\phi(x, \tau), \tag{2}$$
where $\tau = (s_1, s_2, \ldots, s_T, a)$ is the complete chain of reasoning ending in answer a. While simple, ORMs suffer from severe credit assignment ambiguity, unable to distinguish a sound proof from a hallucinated derivation landing on the correct number through compensating arithmetic errors.
2. **Process Reward Models (PRM):** Pioneered by Lightman et al. (2023) in *"Let's Verify Step by Step"*, PRMs assign an individual reward to each intermediate step:
$$r_t = f_\phi(x, s_{1:t-1}, s_t) \in \{-1.0, +1.0\}, \quad t \in \{1, \ldots, T\}. \tag{3}$$
By providing dense, step-level process supervision, PRMs identify the exact step where an error originates, preventing error propagation and enabling credit assignment during search.

Local Validity Semantics: Under step-level PRM semantics, the judge evaluates the local validity of the current step **assuming all preceding steps are correct**. That is, a step is judged `VALID`

if its arithmetic is sound and it follows logically from the problem statement combined with the preceding steps as written. An error in an earlier step does not by itself make subsequent steps invalid if their local arithmetic and deductions remain consistent with the prior text.

2.3 Why Binary Step Verification?

In this assignment, our PRM verifier classifies each step with a **binary judgment**:

$$v_t \in \{\text{VALID}, \text{INVALID}\}. \tag{4}$$

While continuous ratings (e.g., 1 to 5) are common in conversational evaluations (such as PA2's pairwise chat preference judges), multi-step mathematical reasoning possesses a critical mathematical property: **soundness is non-compensatory (all-or-nothing)**. A mathematical argument cannot be "80% correct" if it divides by zero at Step 2; a single invalid deduction invalidates the entire downstream proof.

Empirically, prompted LLM judges on numeric 1 to 5 scales suffer from severe calibration drift (e.g., clustering around 4, or treating a score of 3 inconsistently). A discrete categorical decision with a required concise justification forces the judge to evaluate logical soundness strictly against preceding steps.

2.4 Path Score Aggregation Strategies

Given a reasoning path $\tau = (s_1, s_2, \ldots, s_T)$ and corresponding step scores $(r_1, r_2, \ldots, r_T)$ where $r_t \in \{-1.0, +1.0\}$, students must design and evaluate rules that aggregate these step scores into a scalar path score $S(\tau)$.

1. **Minimum-Pooling (Weakest Link):**
$$S_{\min}(\tau) = \min_{t \in \{1,\ldots,T\}} r_t \tag{5}$$
Reflects the principle that a chain of reasoning is only as strong as its weakest link. If any step is invalid ($r_t = -1.0$), the path score drops to -1.0.
2. **Product / Joint Probability Formulation:** Assuming the verifier outputs estimated validity probabilities $p_t \in (0, 1)$, the path score represents joint validity:
$$S_{\text{prod}}(\tau) = \prod_{t=1}^{T} p_t. \tag{6}$$
With binary scores $r_t \in \{-1.0, +1.0\}$, mapping to clamped probabilities $p_t = \text{clamp}((r_t + 1)/2, 0.01, 0.99)$ yields $p_t \in \{0.01, 0.99\}$. The product $\prod_t p_t$ ranks paths first by the number of invalid steps (each invalid step multiplies by 0.01), and among all-valid paths, mildly favors shorter proofs (0.99^T decreases slightly with length T). Students should expect product aggregation to behave similarly to minimum-pooling and explain differences observed in their report.
3. **Step Error Penalty:**
$$S_{\text{penalty}}(\tau) = \frac{1}{T}\sum_{t=1}^{T} r_t - \lambda \cdot \sum_{t=1}^{T} \mathbb{I}(r_t = -1.0), \tag{7}$$
where $\lambda \geq 0$ is a penalty parameter (default $\lambda = 2.0$). For a completely sound path where all $r_t = +1.0$, the mean is 1.0 at any length. When invalid steps occur, the score is penalized both by decreasing the mean and by subtracting λ per error.
4. **Optional Bonus (+3 Points, Total Capped at 100):** In `src/`, students may optionally implement continuous scoring `"product_logprob"`, where $p_t = \frac{\exp(z_{\text{VALID}})}{\exp(z_{\text{VALID}})+\exp(z_{\text{INVALID}})}$ is computed from the judge model's raw logits at the verdict token, giving continuous step confidence without retraining. Note that `"VALID"` and `"INVALID"` may tokenize into different multi-token sequences; students may compare first-token logits or full-sequence log-probabilities, and must state which approach was used in the report.

3 Model Choice & Compute Environment

3.1 The Local Track: Asymmetric Policy-Verifier Pair

Rather than using a single model for both tasks, PA3 deploys an **asymmetric policy-verifier architecture**:

- **Generator (Policy):** `Qwen/Qwen2.5-1.5B-Instruct`, loaded in full **float16** precision (no quantization; ≈ 3.1 GB VRAM).
- **PRM Judge (Verifier):** `Qwen/Qwen2.5-7B-Instruct`, loaded in **4-bit NormalFloat (NF4)** via `bitsandbytes` (≈ 5.5 GB VRAM).

Total GPU memory footprint is $\approx 3.1\,\text{GB} + 5.5\,\text{GB} \approx 8.6$–$9.0$ GB VRAM.

3.1.1 Technical Rationale

1. **Restored Performance Headroom on GSM8K:**
   Qwen reports GSM8K greedy accuracy in the low 90s for `Qwen2.5-7B-Instruct`, which leaves too little headroom on 50 problems for search to demonstrate substantial value. Conversely, Qwen reports roughly the low 70s for `Qwen2.5-1.5B-Instruct`, which restores the intended headroom (our reference implementation reaches roughly 60–70% greedy accuracy in the required step format). At this baseline, generating 5 diverse paths regularly uncovers correct solutions where greedy decoding fails.
2. **Stronger Verifier than Policy:**
   Using a larger 7B model to judge a smaller 1.5B generator mirrors frontier PRM setups and is expected to reduce self-verification bias (a model judging its own outputs shares its own blind spots and hallucinations).
3. **Strict Delimiter Compliance Across the Qwen2.5 Family:**
   Qwen2.5 models exhibit industry-leading prompt adherence, reliably emitting numbered steps (`Step 1: ...`) and clean answer tags (`Final Answer: ...`), drastically reducing regex parsing failures.
4. **Hardware Feasibility on Kaggle GPU T4x2 (≈ 9 GB Total VRAM):**
   Loading the 1.5B generator in float16 consumes ≈ 3.1 GB, while loading the 7B judge in 4-bit NF4 consumes ≈ 5.5 GB. The two models easily reside simultaneously across Kaggle's dual T4 GPUs without memory pressure.

3.2 Clone the Starter Repository

The student-facing starter repository is hosted on GitHub:

https://github.com/ahmad-sarfraz345/cs5326-pa3-starter.git

Clone the repository to your local development machine or directly into your Kaggle workspace:

```
git clone https://github.com/ahmad-sarfraz345/cs5326-pa3-starter.git
cd cs5326-pa3-starter
```

The starter repository provides the fixed 50-problem GSM8K dataset, public smoke tests, starter code skeletons with type hints and docstrings under `src/`, the grading interface scaffold `submission_adapter.py`, and the packaging script `make_submission.sh`.

3.3 Starter Repository Layout

Your repository should follow this clean layout:

```
cs5326-pa3-starter/
|-- data/
|   `-- gsm8k_test_50.jsonl          # Fixed 50-problem evaluation benchmark
```

```
|-- src/
|   |-- __init__.py
|   |-- generation.py                # Step prompt, greedy, and self-consistency
|   |-- extraction.py                # Regex parser for steps and numerical answers
|   |-- prm_judge.py                 # Step-level verifier prompt & scoring loop
|   |-- search.py                    # PRM-guided Best-of-N and budget tracking
|   `-- utils.py                     # Dual model loading, logging, bookkeeping
|-- tests/
|   |-- __init__.py
|   `-- test_public_adapter_smoke.py # Public smoke test suite for grading interface
|-- outputs/                         # Generated predictions and metric dumps
|   |-- per_problem.jsonl            # Detailed per-problem outputs and step scores
|   |-- results_summary.json         # Automated metrics table across all configurations
|   |-- aggregation_ablation.json    # Aggregation strategy comparison table
|   `-- sc_scaling_curve.json        # Self-consistency accuracy for N = 1 ... 15
|-- run_pipeline.ipynb               # Top-to-bottom runnable evaluation notebook (or run_all.py)
|-- submission_adapter.py            # Grading adapter contract (Required)
|-- requirements.txt
|-- pyproject.toml
|-- make_submission.sh               # Submission bundler script
|-- REPORT_TEMPLATE.md               # Scaffold for written report
`-- REPORT.pdf                       # Final technical report (Max 6 pages)
```

3.4 Kaggle Setup and Environment

You are strongly recommended to complete this assignment on a free **Kaggle Notebook** with GPU acceleration.

1. **Accelerator Setting:** In the top sidebar of your Kaggle notebook, set the **Accelerator** to **GPU T4x2**.
2. **Internet Setting:** In notebook settings, ensure **Internet** is toggled to **On** so your session can download packages and model weights.
3. **Cloning into Kaggle:** In the first code cell of your Kaggle notebook, clone the starter repository and switch into the project directory:

   ```
   !git clone https://github.com/ahmad-sarfraz345/cs5326-pa3-starter.git
   %cd cs5326-pa3-starter
   !ls -la
   ```

   Note: All relative paths (e.g., `data/gsm8k_test_50.jsonl`) assume your active working directory is the repository root.
4. **Dependency Installation:** Kaggle's image already provides PyTorch, Transformers, Accelerate, Datasets, NumPy and pytest; do not reinstall or pin them. Only bitsandbytes must be upgraded: older pinned releases (e.g. 0.44.x) fail on current Kaggle images because they import triton.ops, which no longer exists. If a package is reported missing, install it individually and restart the kernel.

   ```
   !pip install -q -U bitsandbytes
   ```

5. **Device Placement Strategy:** Kaggle's **GPU T4x2** environment exposes two physical GPUs. Place the generator on `cuda:0` and the judge on `cuda:1` (using `device_map={"": 0}` and `device_map={"": 1}`). If working on a single GPU, place both models on `cuda:0`.
   Warning: Do **not** use `device_map="auto"` for a single model across two GPUs; splitting a model's layers across PCIe buses causes catastrophic latency overhead during generation. Note that NVIDIA T4 GPUs do not support `bfloat16`, so use `torch.float16`.

Dual-Model Loading Snippet (Implemented in src/utils.py)

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# 1. Generator: Qwen2.5-1.5B-Instruct in float16 on cuda:0 (~3.1 GB VRAM)
gen_id = "Qwen/Qwen2.5-1.5B-Instruct"
gen_tok = AutoTokenizer.from_pretrained(gen_id)
gen_model = AutoModelForCausalLM.from_pretrained(
    gen_id, torch_dtype=torch.float16, device_map={"": 0}
)

# 2. PRM Judge: Qwen2.5-7B-Instruct in 4-bit NF4 on cuda:1 (~5.5 GB VRAM)
judge_id = "Qwen/Qwen2.5-7B-Instruct"
judge_tok = AutoTokenizer.from_pretrained(judge_id)
judge_bnb = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16, bnb_4bit_use_double_quant=True
)
judge_dev = {"": 1} if torch.cuda.device_count() > 1 else {"": 0}
judge_model = AutoModelForCausalLM.from_pretrained(
    judge_id, quantization_config=judge_bnb, device_map=judge_dev
)
```

Batching, Checkpointing, and Sampling Parameters

Batching and Checkpointing:

- Batch candidate sampling using `num_return_sequences=5` or batched input prompts.
- Batch judge prompts across steps and candidates where possible.
- Save intermediate per-problem results incrementally to `outputs/*.jsonl` so execution can resume seamlessly if a Kaggle session disconnects.

Sampling, Decoding, and Seeds:

- **Global Seed:** Call `transformers.set_seed(5326 + problem_index)` before each problem to guarantee reproducibility and resumability.
- **Greedy Baseline & Judge:** Pass `do_sample=False` together with `temperature=None`, `top_p=None`, `top_k=None` to fully override Qwen's shipped generation defaults and silence warnings.
- **Stochastic Sampling (SC & BoN):** Use `temperature=0.7`, `top_p=1.0`, `top_k=0`, `repetition_penalty=1.0`. These parameters deliberately override Qwen's default settings (`top_p=0.8`, `top_k=20`, `repetition_penalty=1.05`) to match pure temperature sampling.
- **Token Limits:** `max_new_tokens=512` for generation, `max_new_tokens=96` for judge decisions.

4 The Benchmark Dataset & Step-Structured Formatting

4.1 The GSM8K 50-Problem Subset

The evaluation dataset is located at `data/gsm8k_test_50.jsonl`. It consists of exactly 50 multi-step grade school math word problems from the official GSM8K test split (Cobbe et al., 2021). Each line is a valid JSON object:

```
{
  "id": "gsm8k_test_01",
  "question": "Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and
    bakes muffins for her friends every day with four. She sells the remainder at the farmers'
    market daily for $2 per fresh duck egg. How much in dollars does she make every day at the
    farmers' market?",
  "ground_truth_answer": 18
}
```

Notice that every problem requires multi-step arithmetic, logic tracking, and concludes in a single integer answer.

4.2 Step-Structured Prompt Schema

To score intermediate reasoning steps with a Process Reward Model, the generator cannot output a free-form unbroken paragraph. It must emit discrete, numbered deductions ending with a canonical answer delimiter.

You will construct a prompt template that enforces the following output format:

```
Step 1: <First deduction or arithmetic operation>
Step 2: <Second deduction conditioning on Step 1>
...
Step K: <Final calculation>
Final Answer: <Integer>
```

Format Compliance Warning

In our reference runs, Qwen2.5-1.5B under greedy decoding frequently drifted into markdown headings, LaTeX, or `\boxed{}` and omitted the "`Final Answer:`" line, which the strict parser correctly scores as `None`. Your prompt must explicitly prevent this; check your greedy delimiter-compliance rate on all 50 problems before launching the full run.

Example Compliant Generation

Question: Janet's ducks lay 16 eggs per day. She eats 3 and bakes with 4. She sells the rest for $2 each. How much does she make?

Model Completion:

```
Step 1: Janet starts with 16 eggs per day.
Step 2: She uses 3 eggs for breakfast and 4 eggs for muffins, which is 3 + 4 = 7 eggs used.
Step 3: The remaining eggs to sell are 16 - 7 = 9 eggs.
Step 4: She sells each remaining egg for $2, so she makes 9 * 2 = 18 dollars.
Final Answer: 18
```

4.3 Step and Answer Extraction Specifications

LLM completions in practice often contain markdown bolding (e.g., `**Step 1:**`), arbitrary whitespace, or variations in punctuation. Your step parser must be resilient to these formatting artifacts:

```
import re
```

```python
def parse_steps(completion_text: str) -> list[str]:
    """Extracts step text blocks tolerant to markdown headings (### Step 2:) and bolding."""
    pattern = r"(?:^|\n)\s*(?:[#*`\s]*Step\s*\d+\s*[:.]?[#*`\s]*)(.*?)(?=(?:\n\s*[#*`\s]*Step\s*\
    d+\s*[:.]?[#*`\s]*)|(?:\n\s*[#*`\s]*Final Answer)|$)"
    matches = re.findall(pattern, completion_text, flags=re.DOTALL | re.IGNORECASE)
    cleaned = []
    for m in matches:
        text = m.strip().strip("*#`").strip()
        if text:
            cleaned.append(text)
    return cleaned
```

Listing 1: Markdown-Tolerant Step Parser Skeleton (`src/extraction.py`)

Answer Parsing Robustness: The answer parser (`extract_answer`) must never crash on malformed outputs, and must correctly extract integer answers across diverse generation patterns:

Completion Snippet	Target	Rationale
`Final Answer: 18`	18	Standard exact format.
`Final Answer: $1,234`	1234	Strips currency signs and thousand commas.
`Final Answer: 18.00`	18	Accepts integer-valued decimals.
`Final Answer: 18.5`	None	Non-integer values fail extraction.
`Final Answer: -5`	-5	Preserves negative signs.
`Final Answer: 18 dollars.`	18	Strips trailing units and sentence punctuation.
`**Final Answer:** 18`	18	Handles markdown bold decoration around delimiter.
`final answer 18`	18	Case-insensitive matching with optional colon.
Multiple `Final Answer` lines	Use last	Discards scratchpad attempts, selecting final line.
No `Final Answer` present	None	Strictly returns None if delimiter missing.
Empty string or malformed garbage	None	Returns None and must never raise exceptions.

There is deliberately no fallback heuristic to arbitrary integers: format failures should surface explicitly as None and be analyzed in report Section 3.

5 Task Specifications & Problem Deliverables

5.1 Problem 1: Step-Structured Generation & Baseline Decoders (20 Points)

Problem 1: Step-Structured Generation & Baselines (20 Points)

Deliverable 1.1: Prompt Formatting & Answer Extraction (5 pts)
Implement `format_step_prompt(question)`, `parse_steps(text)`, and `extract_answer(text)` in `src/generation.py`, `src/extraction.py`, and `submission_adapter.py`. Your prompt must instruct the model to produce numbered steps followed by `Final Answer: <int>`. The extraction functions must never crash on malformed outputs, returning `None` when no answer is found.

Deliverable 1.2: Greedy Baseline Decoding ($N = 1$) (5 pts)
Implement `run_greedy(model, tokenizer, question, generate_fn=None)` in `src/generation.py` and `submission_adapter.py`. Generate a single completion with `do_sample=False` and explicit `temperature=None, top_p=None, top_k=None`. Return a dictionary containing `"prompt"`, `"completion"`, `"steps"`, `"answer"`, `"generation_calls"`, `"generated_tokens"`, and `"time_seconds"`.

Deliverable 1.3: Self-Consistency Majority-Voting Baseline ($N = 5$) (10 pts)
Implement `run_self_consistency(model, tokenizer, question, n=5, temperature=0.7, generate_fn=None)` in `src/generation.py` and `compute_majority_vote(answers)` in `submission_adapter.py`. Sample $N = 5$ inde-

pendent reasoning trajectories per problem using `temperature=0.7, top_p=1.0, top_k=0, repetition_penalty=1.0`. Extract candidate answers and aggregate via **majority vote**:

$$\hat{a}_{\text{SC}} = \arg\max_{a} \sum_{i=1}^{N} \mathbb{I}(a^{(i)} = a). \tag{8}$$

Ignore `None` values. Break ties deterministically in favor of the answer appearing earlier in the candidate sequence. Return `"candidates"` (each containing `completion`, `steps`, `answer`, `num_tokens`), `"answer"`, `"generation_calls"`, `"generated_tokens"`, and `"time_seconds"`. If no candidate produces a valid integer, the returned `"answer"` is `None`.

5.2 Problem 2: Process Reward Model (PRM) Verifier (25 Points)

Problem 2: PRM Step-Level Verifier & Aggregation (25 Points)

Deliverable 2.1: Step-Level Judge Prompting & Local Validity (10 pts)
Implement `build_prm_judge_prompt(question, preceding_steps, current_step)` and `score_path(judge_model, judge_tokenizer, question, candidate, judge_fn=None)` in `src/prm_judge.py` and `submission_adapter.py`. The candidate parameter is a dictionary containing key `"steps"`.
The prompt must instruct the judge (`Qwen2.5-7B-Instruct`) to evaluate `current_step` under **local validity semantics** (assuming preceding steps are correct). The autograder verifies the prompt structure offline using mocks by checking the following exact literals:

- Preceding steps are rendered as `"Step 1: <text>"`, `"Step 2: <text>"`, ... in order.
- The current step is rendered with the label `"Current Step:"`.
- The prompt contains the phrase `"assume all preceding steps are correct"` (case-insensitive).
- The prompt contains the literal strings `"Verdict: VALID"`, `"Verdict: INVALID"`, and `"Rationale:"`.
- With an empty `preceding_steps` list, the prompt states `"(no preceding steps)"`.

Deliverable 2.2: Path Score Aggregation Rules (10 pts)
Implement `aggregate_path_score(step_scores, strategy="min", penalty_lambda=2.0)` in `src/search.py` and `submission_adapter.py`.

- An empty list of step scores returns -1.0 for all strategies.
- `"min"`: $\min(step_scores)$.
- `"product"`: Convert scores to clamped probabilities $p_t = \text{clamp}((r_t+1)/2, 0.01, 0.99)$ and return the raw product $\prod_t p_t$ (not the log).
- `"step_penalty"`: $\text{mean}(step_scores) - \lambda \cdot \sum_t \mathbb{I}(r_t = -1.0)$ with default $\lambda = 2.0$.
- An unknown strategy must raise `ValueError`.

Deliverable 2.3: Verification Resiliency & Parser (5 pts)
Implement `parse_judge_decision(judge_output) -> tuple[float, str]`. Rules: (1) If a line matches `/verdict\s*:\s*\**\s*(VALID|INVALID)\b/` (case-insensitive), with the first match winning. (2) Otherwise, if the standalone word `"INVALID"` or phrase `"NOT VALID"` appears (case-insensitive, word boundaries), return -1.0. (3) Otherwise, if the standalone word `"VALID"` appears, return $+1.0$. (4) Otherwise, default to -1.0 (unparseable defaults to INVALID). **Warning:** A naive `"VALID" in text` check is wrong because `"INVALID"` contains

`"VALID"`. Extract the one-sentence rationale after `"Rationale:"` up to the first sentence end (`"."`, `"!"`, or `"?"` followed by whitespace or end of string, so decimals like `"2.5"` are not split) or newline, whichever comes first, stripped; empty string if absent.

5.3 Problem 3: PRM-Guided Best-of-N Search & Budget Bookkeeping (20 Points)

Problem 3: PRM-Guided Best-of-N & Budget Matching (20 Points)

Deliverable 3.1: PRM-Guided Best-of-N Selection ($N = 5$) (8 pts)
Implement `select_best_path(candidate_paths, aggregation_strategy="min", penalty_lambda=2.0)` and `run_prm_best_of_n(model, judge_model, gen_tokenizer, judge_tokenizer, question, n=5, aggregation_strategy="min", penalty_lambda=2.0, candidates=None, generate_fn=None, judge_fn=None)` in `src/search.py` and `submission_adapter.py`. Each candidate dictionary in `candidate_paths` must contain `"step_scores"` (`list[float]`) and `"answer"` (`int | None`); other keys are passed through unchanged in the returned copy. If `candidates` is provided to `run_prm_best_of_n`, they use the same format as `run_self_consistency` candidates (at minimum `"steps"` and `"answer"`), and the function does not sample new candidates (`generation_calls=0`, `generated_tokens=0`). Returns: `selected`, `answer`, `candidates` (augmented with `step_scores` and `rationales`), `generation_calls`, `judge_calls`, `generated_tokens`, `judge_tokens`, and `time_seconds`.

1. Compute `path_score` for each candidate using `aggregate_path_score`. Empty steps yield -1.0.
2. **Eligibility:** If at least one candidate has a non-`None` answer, candidates with `answer=None` are excluded. If all answers are `None`, all candidates remain eligible.
3. Find the maximum `path_score` among eligible candidates (tolerance 10^{-9}). Candidates within tolerance form the *tied set*.
4. **Tie-Breaking via Majority Vote:** If multiple candidates tie, compute `compute_majority_vote` over their answers. Select the lowest-index candidate in the tied set whose answer equals the vote. If the vote is `None`, return the lowest-index tied candidate.
5. Return a shallow **copy** of the chosen candidate dict with added keys `"path_score"` (float) and `"num_tied"` (int). Do not mutate input dictionaries.

Why this matters: Under binary min-pooling, multiple candidates frequently tie at $+1.0$. A naive "first candidate wins" rule reduces PRM search to a single random sample. The filter-then-vote rule lets the PRM eliminate flawed paths, while majority voting chooses among surviving sound candidates.

Deliverable 3.2: Compute Bookkeeping (6 pts)
Track and record computational cost adhering strictly to the bookkeeping rules box below.

Deliverable 3.3: Compute-Matched Self-Consistency ($N_{\text{SC}} \approx 15$) (6 pts)
Implement `run_compute_matched_self_consistency(model, tokenizer, question, target_calls=15, temperature=0.7, existing_candidates=None, generate_fn=None)` in `src/search.py` and `submission_adapter.py`. If `existing_candidates` is provided, sample only the deficit (`target_calls - len(existing_candidates)`) and report `generation_calls` and `generated_tokens` for new samples only. The returned `"candidates"` list contains `existing_candidates` followed by the newly sampled ones, in sampling order. Returns: `candidates`, `answer`, `generation_calls`, `total_samples`, `generated_tokens`, and `time_seconds`.
PRM Best-of-5 consumes 5 generation calls plus $5 \times \bar{K}$ judge calls (where $\bar{K} \approx 4$–6 is the mean number of steps per path on GSM8K), totaling $\approx$ 25–35 total LLM calls. Setting $N_{\text{SC}} = 15$ provides a fixed, approximately call-matched sampling baseline. Because generation calls and judge calls have asymmetric token costs (judge calls have long prompts and short outputs; generation calls are the reverse), report both calls and tokens.

Compute Bookkeeping Rules

- `generation_calls`: Exactly one generated sequence equals one generation call, regardless of batching (e.g., `num_return_sequences=5` counts as 5 generation calls).
- `judge_calls`: Exactly $\sum_{i=1}^{N} K_i$ where K_i is the number of parsed steps evaluated for candidate i.
- `generated_tokens`: Output completion tokens generated across candidates (excluding prompt tokens).
- `judge_tokens`: Output completion tokens generated by the judge (excluding prompt tokens).
- Token counts must be computed from token IDs (`len(output_ids[len(input_ids):])`), not string splitting.

Candidate Pool Sharing for Direct Comparison

For a controlled evaluation, **Self-Consistency ($N = 5$) and all three PRM Best-of-5 variants (min, Product, Step Penalty) must evaluate the exact same candidate pool of 5 sampled paths**.

- Sample 5 diverse paths once per problem.
- Apply majority voting over their answers for Self-Consistency ($N = 5$).
- Score those identical 5 paths with the PRM judge. Reuse the resulting step scores across min, Product, and Step Penalty without re-running the judge!
- For Compute-Matched Self-Consistency ($N_{\text{SC}} = 15$), retain the initial 5 paths (indices 0–4) and sample 10 additional completions (indices 5–14).
- For each $N \in \{1, \ldots, 15\}$, compute the self-consistency scaling curve by taking the majority vote over the **first N samples in sampling order**. By construction, the $N = 5$ point must equal the Self-Consistency ($N = 5$) row, and the $N = 15$ point must equal the Compute-Matched SC row.

5.4 Problem 4: Evaluation, Comparative Analysis & Report (35 Points)

Problem 4: Evaluation, Metrics & Technical Report (35 Points)

Run all configurations across the entire 50-problem benchmark (`data/gsm8k_test_50.jsonl`):

1. **Greedy Baseline** ($N = 1$)
2. **Self-Consistency Baseline** ($N = 5$, candidate-matched)
3. **Compute-Matched Self-Consistency** ($N_{\text{SC}} = 15$)
4. **PRM Best-of-5** (min-pooling)
5. **PRM Best-of-5** (Product aggregation)
6. **PRM Best-of-5** (Step Penalty aggregation)

Deliverable 4.1: Programmatic Evaluation Artifacts (10 pts)
Your notebook/script (`run_pipeline.ipynb` or `run_all.py`) must automatically evaluate exact-match accuracy against `ground_truth_answer` and serialize all required evaluation artifacts:

1. `outputs/results_summary.json`: Complete 6-row summary table across all methods.
2. `outputs/aggregation_ablation.json`: Direct ablation comparing min, Product, and Step Penalty.
3. `outputs/sc_scaling_curve.json`: Self-consistency accuracy for each sub-budget $N \in \{1, \ldots, 15\}$.
4. `outputs/per_problem.jsonl`: Per-problem completions and predictions (schema as in Section 9).

5. `run_pipeline.ipynb` or `run_all.py`: Complete, reproducible execution notebook or script.

Method	Acc (%)	Gen Calls	Judge Calls	Total Calls	Avg Gen Tok/Prob	Avg Judge Tok/Prob	Time (s)
Greedy (N=1)	-	50	0	50	-	0.0	-
Self-Consistency (N=5)	-	250	0	250	-	0.0	-
Compute-Matched SC (N=15)	-	750	0	750	-	0.0	-
PRM Best-of-5 (min)	-	250	$250 \times \bar{K}$	$250 + 250\bar{K}$	-	-	-
PRM Best-of-5 (product)	-	250	$250 \times \bar{K}$	$250 + 250\bar{K}$	-	-	-
PRM Best-of-5 (step_penalty, lambda=2.0)	-	250	$250 \times \bar{K}$	$250 + 250\bar{K}$	-	-	-

Note: PRM Best-of-5 configurations share identical candidate paths and step scores. In `results_summary.json`, all three PRM rows report the same numeric Generation Calls, Judge Calls, Total LLM Calls, and token columns; judging is shared across rows. Time attribution rules: (1) Greedy = its own generation time; (2) SC@5 = time to sample the 5 shared candidates; (3) Each PRM row (min, product, step_penalty) = SC@5 sampling time + total judging time + its own aggregation/selection time (each row reports the full cost of running that method, even though judging is computed once and cached); (4) SC@15 = SC@5 sampling time + time to sample the 10 additional candidates.

The serialized `outputs/` files must strictly adhere to these exact schemas:

- `results_summary.json`: List of 6 objects matching the method strings above with keys `"Method"`, `"Accuracy (%)"`, `"Generation Calls"`, `"Judge Calls"`, `"Total LLM Calls"`, `"Avg Gen Tokens / Problem"`, `"Avg Judge Tokens / Problem"`, `"Total Time (s)"`.
- `aggregation_ablation.json`: List of `{"Strategy", "Lambda", "Accuracy (%)", "Tie Rate (%)"}` for min, Product, and Step Penalty at $\lambda \in \{0.0, 0.5, 2.0\}$ (`Lambda` is `null` for min and Product).
- `sc_scaling_curve.json`: List of `{"N", "Accuracy (%)", "Avg Gen Tokens / Problem"}` for $N = 1, \ldots, 15$, evaluated via majority voting over the first N samples in sampling order (indices 0 to $N-1$). Consequently, $N = 5$ equals the SC@5 row and $N = 15$ equals the Compute-Matched SC row.

Deliverable 4.2: Technical Report Analysis (25 pts)
Submit a written PDF report (`REPORT.pdf`, maximum 6 pages). See Section 7 for the mandatory 5-section report structure and analytical requirements.

6 Private Evaluation Interface: Submission Adapter Contract

To enable automated testing without dictating internal package names or module structure, you must provide a root-level file named `submission_adapter.py`.

The automated grading harness will import this file and execute unit tests against mock models and synthetic reasoning traces. Every function must strictly adhere to the signatures below.

```python
"""
CS 5326: Advanced Generative AI and Agents - Programming Assignment 3
Submission Adapter Interface Contract.
"""
from typing import Any, Callable

# Type aliases for mockable pipeline callables (offline testing & autograding)
GenerateFn = Callable[[str, int, dict], list[dict]]
# generate_fn(raw_prompt, num_samples, gen_kwargs) -> list of {"text": str, "num_tokens": int}
# gen_kwargs carries do_sample/temperature/top_p/top_k/repetition_penalty/max_new_tokens

JudgeFn = Callable[[str], dict]
# judge_fn(judge_prompt) -> {"text": str, "num_tokens": int}

def format_step_prompt(question: str) -> str:
    """Formats question into prompt enforcing numbered steps and 'Final Answer: <int>'."""
    ...

def parse_steps(completion_text: str) -> list[str]:
    """Parses completion text and returns a list of individual step strings."""
    ...

def extract_answer(completion_text: str) -> int | None:
    """Extracts final integer from completion. Returns None if unparseable."""
    ...

def compute_majority_vote(answers: list[int | None]) -> int | None:
    """Majority vote over answers, ignoring None and breaking ties by earliest index."""
    ...

def build_prm_judge_prompt(question: str, preceding_steps: list[str], current_step: str) -> str:
    """Builds verification prompt under local validity semantics."""
    ...

def parse_judge_decision(judge_output: str) -> tuple[float, str]:
    """Parses judge completion into (score, rationale). Score is +1.0 (VALID) or -1.0 (INVALID)."
    ""
    ...

def aggregate_path_score(step_scores: list[float], strategy: str = "min", penalty_lambda: float =
     2.0) -> float:
    """Aggregates step scores into a path score. Supports 'min', 'product', 'step_penalty'."""
    ...

def select_best_path(
    candidate_paths: list[dict[str, Any]],
    aggregation_strategy: str = "min",
    penalty_lambda: float = 2.0
) -> dict[str, Any]:
    """Scores candidate paths, excludes answer=None if valid answers exist, and returns
    a shallow copy of the best path with 'path_score' and 'num_tied', breaking ties via majority
    vote."""
    ...

def run_greedy(
    model: Any = None, tokenizer: Any = None, question: str = "",
    generate_fn: GenerateFn | None = None
```

```python
) -> dict[str, Any]:
    """Returns: prompt, completion, steps, answer, generation_calls, generated_tokens,
    time_seconds"""
    ...

def run_self_consistency(
    model: Any = None, tokenizer: Any = None, question: str = "", n: int = 5,
    temperature: float = 0.7, generate_fn: GenerateFn | None = None
) -> dict[str, Any]:
    """Returns: candidates [each: completion, steps, answer, num_tokens], answer,
    generation_calls, generated_tokens, time_seconds"""
    ...

def score_path(
    judge_model: Any = None, judge_tokenizer: Any = None, question: str = "",
    candidate: dict[str, Any] | None = None, judge_fn: JudgeFn | None = None
) -> dict[str, Any]:
    """Must call build_prm_judge_prompt and parse_judge_decision. candidate has key 'steps'.
    Returns: step_scores, rationales, judge_calls, judge_tokens"""
    ...

def run_prm_best_of_n(
    model: Any = None, judge_model: Any = None, gen_tokenizer: Any = None, judge_tokenizer: Any =
     None,
    question: str = "", n: int = 5, aggregation_strategy: str = "min", penalty_lambda: float =
    2.0,
    candidates: list[dict[str, Any]] | None = None,
    generate_fn: GenerateFn | None = None, judge_fn: JudgeFn | None = None
) -> dict[str, Any]:
    """If candidates given, does not sample (generation_calls=0, generated_tokens=0).
    Provided candidates use the run_self_consistency candidate format (at minimum 'steps' and '
    answer').
    Returns: selected, answer, candidates [each augmented with step_scores, rationales],
    generation_calls, judge_calls, generated_tokens, judge_tokens, time_seconds"""
    ...

def run_compute_matched_self_consistency(
    model: Any = None, tokenizer: Any = None, question: str = "", target_calls: int = 15,
    temperature: float = 0.7, existing_candidates: list[dict[str, Any]] | None = None,
    generate_fn: GenerateFn | None = None
) -> dict[str, Any]:
    """Samples target_calls - len(existing_candidates) new paths.
    Returned candidates = existing_candidates followed by new samples, in sampling order.
    Returns: candidates, answer, generation_calls (new samples only), total_samples,
    generated_tokens (new samples only), time_seconds"""
    ...
```

Listing 2: submission_adapter.py interface definition

6.1 Running the Public Adapter Tests

Before packaging your submission, verify that your adapter functions conform to the test contract using the included public smoke tests. You can run them using either standard Python or `pytest` without any test framework dependencies:

```bash
# Option A: Run via standard Python
python tests/test_public_adapter_smoke.py

# Option B: Run via pytest
pytest -v tests/test_public_adapter_smoke.py
```

The smoke test suite runs entirely offline without downloading models or needing a GPU. It verifies that parsing correctly extracts tricky multi-digit numbers, that majority voting resolves ties deterministically, that aggregation functions compute correct values across boundary edge cases, and that end-to-end mock-pipeline tests (greedy decoding, self-consistency, score_path, Best-of-N, compute-matched SC, and bookkeeping counts) execute with 100% contract fidelity.

7 Technical Report Specifications

Your final report (`REPORT.pdf`) must be at most **6 pages** long (including figures, tables, and references) and must be organized into the following five sections:

7.1 Section 1: Benchmark Results and Scaling Curves (7 Points)

- Present the complete comparative table of results on the 50-problem benchmark across Greedy ($N = 1$), Self-Consistency ($N = 5$), Compute-Matched Self-Consistency ($N_{\text{SC}} = 15$), and PRM Best-of-5 (min, Product, Step Penalty).
- Present two scaling plots: (1) **Accuracy vs. Total LLM Calls** (Self-Consistency $N = 1 \ldots 15$ with PRM points overlaid); (2) **Accuracy vs. Total Output Tokens** (generated tokens + judge tokens).
- Report **paired win/loss counts** (PRM vs. SC-5, PRM vs. Compute-Matched SC-15).
- Compute **95% bootstrap confidence intervals** for accuracy across methods (1,000 resamples over problems, seed 5326).
- Discuss whether observed accuracy differences between PRM search and sampling baselines exceed statistical noise.

7.2 Section 2: Path Score Aggregation Ablation (6 Points)

- Present the ablation results from `outputs/aggregation_ablation.json` comparing min, Product, and Step Penalty at $\lambda \in \{0.0, 0.5, 2.0\}$ along with measured **tie rates**.
- Analyze the mathematical dynamics of binary step scores under each aggregation rule.
- Analyze the **filter-then-vote interaction**: how the majority-vote tie-breaking rule interacts with min-pooling when multiple candidate trajectories achieve a score of $+1.0$.
- If you implemented the optional continuous logprob bonus, discuss whether true token probabilities improved ranking over discrete decisions.

7.3 Section 3: Failure Mode and Judge Error Analysis (7 Points)

- Select 3 concrete failure cases: Judge False Positive (flawed step marked VALID), Judge False Negative (valid step marked INVALID), and Mode Collapse / Shared Flawed Premise. If a failure category does not occur in your run, present the closest example and explain its absence.
- **Required Judge Audit:** Hand-label at least **30 individual judged steps** (including at least **10 steps marked INVALID** by the judge). Report the confusion matrix (TP, FP, TN, FN) and judge precision/recall for INVALID detection. If any error category is absent or cell is zero, explain why based on model error rates.
- **Discussion Question:** How does utilizing a substantially stronger judge model (7B) than generator policy (1.5B) influence the types of reasoning errors caught, and what failure modes would you anticipate if the 1.5B generator were tasked with self-verification?

7.4 Section 4: Test-Time Compute Efficiency Analysis (3 Points)

- **Defining 10x Greedy Compute:** Calculate your exact token budget defined as $10\times$ greedy generated tokens. On the spending side, count both generated tokens and judge tokens.
- Quantify compute overhead and structural asymmetry between generator calls and judge calls.
- Under this fixed budget, would you recommend spending compute on **pure sampling (Self-Consistency with large N)** or **PRM search**? Provide empirical justification using your scaling curve.

7.5 Section 5: Reproducibility and Limitations (2 Points)

- Document hardware specifications (Kaggle GPU T4x2 or local GPU), total runtime duration, random seed protocol (`5326 + problem_index`), and library versions.
- Discuss the fundamental theoretical limitation of a *prompted LLM stand-in verifier* compared to a true value function or PRM trained on human step-level supervision (e.g., PRM800K).

8 Grading Rubric Breakdown

The assignment is graded on a **100-point scale**:

Component	Weight	Deliverable	Key Grading Criteria
Problem 1: Generation & Baselines	20 pts	• Deliv 1.1: Prompts & Parser (5) • Deliv 1.2: Greedy Baseline (5) • Deliv 1.3: Self-Consistency (10)	Prompt strictly enforces step schema; regex parser never crashes and handles markdown bolding and commas; majority voting handles ties deterministically.
Problem 2: PRM Judge & Scoring	25 pts	• Deliv 2.1: Judge & Validity (10) • Deliv 2.2: Aggregation Rules (10) • Deliv 2.3: Decision Parsing (5)	PRM prompt adheres to local validity semantics; aggregation handles min, product, penalty, and lambda; parser avoids substring traps and defaults safely.
Problem 3: Best-of-N & Budget	20 pts	• Deliv 3.1: Best-of-N Search (8) • Deliv 3.2: Bookkeeping (6) • Deliv 3.3: Budget-Matched SC (6)	Best-of-N selects highest-scoring path with filter-then-vote tie breaking; accurate tracking of calls and tokens; matched-budget SC runs at $N_{\text{SC}} = 15$.
Problem 4: Programmatic Artifacts	10 pts	• `results_summary.json` • `aggregation_ablation.json` • `sc_scaling_curve.json` • `per_problem.jsonl` • `run_pipeline.ipynb` (or `.py`)	Exact-match evaluation run top-to-bottom across all 50 problems without manual editing; required JSON artifacts serialized cleanly; TAs may spot-check artifacts and report claims against `outputs/per_problem.jsonl`.
Technical Report: Written Analysis	25 pts	• Sec 1: Results & Scaling (7) • Sec 2: Aggregation Ablation (6) • Sec 3: Qualitative Errors (7) • Sec 4: Compute Efficiency (3) • Sec 5: Reproducibility (2)	Substantive discussion; scaling plots with bootstrap CIs and paired counts; λ sweep analysis; REQUIRED judge audit (confusion matrix, precision/recall); 3 annotated failure cases; top-to-bottom reproducibility.
Optional Bonus: Continuous Logprob PRM	+3 pts	• Softmax Logprob in `src/`	Extracts normalized token probabilities from judge logits for continuous step scoring (total assignment score capped at 100).
Total	**100 pts**	**Auto-graded: 65 pts** | **Artifacts: 10 pts** | **Report: 25 pts**	

9 Submission Instructions

From the root of your repository, execute the packaging script by passing your student roll number:

```
bash make_submission.sh <roll_number>
# Example: bash make_submission.sh 24100123
```

This runs the public smoke tests to verify test-contract integrity, packages the required artifacts, and generates a structured archive:

`<roll_number>_pa3.zip` (e.g., `24100123_pa3.zip`)

Upload this zip file directly to the course LMS. The archive must contain:

```
<roll_number>_pa3.zip
|-- src/
|   |-- generation.py
|   |-- extraction.py
|   |-- prm_judge.py
|   |-- search.py
|   `-- utils.py
|-- submission_adapter.py
|-- run_pipeline.ipynb (or run_all.py)
|-- outputs/
|   |-- per_problem.jsonl
|   |-- results_summary.json
|   |-- aggregation_ablation.json
|   `-- sc_scaling_curve.json
`-- REPORT.pdf
```

Note: `outputs/per_problem.jsonl` contains one line per problem with its id, ground truth, greedy output, all 15 candidates, step scores and rationales for the 5 shared candidates, and each method's selected answer. `run_pipeline.ipynb` (or `run_all.py`) must be runnable top to bottom.

10 Academic Integrity

You are encouraged to discuss conceptual ideas, search algorithms, and prompting strategies with peers. However, **all code, prompts, regex parsers, and written report analyses must be entirely your own work**.

- Do not share your `submission_adapter.py` or source code with any classmate.
- Do not copy solutions from public GitHub repositories or pre-packaged agent frameworks.
- All submissions will be checked against an automated code and text similarity detector.

11 References

1. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training Verifiers to Solve Math Word Problems.* arXiv:2110.14168.
2. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). *Let's Verify Step by Step.* arXiv:2305.20050.
3. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* In *International Conference on Learning Representations (ICLR).* arXiv:2203.11171.
4. Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters.* arXiv:2408.03314.
5. Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., & Mirhoseini, A. (2024). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling.* arXiv:2407.21787.
6. Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling Laws for Neural Language Models.* arXiv:2001.08361.
7. Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., et al. (2022). *Training Compute-Optimal Large Language Models.* In *Advances in Neural Information Processing Systems (NeurIPS).* arXiv:2203.15556.
8. Qwen Team (2024). *Qwen2.5 Technical Report.* arXiv:2412.15115.
9. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* In *Advances in Neural Information Processing Systems (NeurIPS).*